

Reddit Virality & Controversy Analysis

What makes a Reddit post go viral — positivity, negativity, or controversy?

DSAN 6000 — Big Data Analysis at Scale
Final Project Report — Group 8

Team Members

Nikhilesh Narendra Dhavale
Aditya Karande
Anushka Rajesh Salvi

May 2026

Executive Summary

This report investigates what makes a Reddit post go viral, drawing on a 446 GB dataset of approximately 500 million submissions and roughly 3 billion comments collected between June 2023 and July 2024. The analysis is organized around a single question: when a post takes off, is it because the content is positive, negative, or controversial, and can we predict that outcome from features available at the moment a post is created?

All processing was carried out on a four-node Apache Spark cluster running on AWS EC2 (t3.large instances), with data stored as Parquet on S3. The workload was deliberately structured so that no single-machine tooling — pandas in particular — could produce the answers; every aggregation, NLP inference pass, and machine-learning model was trained and evaluated through PySpark and Spark MLlib running in distributed mode.

Across ten questions spanning exploratory analysis, natural language processing, and predictive modeling, the project produces a consistent picture. Reddit engagement follows an extreme power law: a vanishing minority of posts captures a disproportionate share of attention, and the rest are effectively invisible. Within that environment, virality is driven less by content alone than by an interaction of community context, timing, language, and emotional tone. Negative and controversial content can drive heavy engagement in the right subreddits, but the formula is not universal — what works on r/AskReddit is not what works on r/MadeMeSmile. Predictive models built only from post-time metadata recover a meaningful but limited share of the signal: roughly twenty percent of variation in comment counts and modest but real discriminative power for both viral and controversial classifiers.

Beyond the substantive findings, the project demonstrates that big-data tooling is not optional for this scale of analysis. The dataset cannot be opened on a single machine; even sampled subsets push the limits of in-memory processing. The Spark cluster, partitioned Parquet storage, and distributed NLP pipelines together make the analysis tractable and reproducible. Where models perform less well at scale than on a single-month sample — most notably the linear regression in Question 8, whose R^2 collapses on the full dataset — that drop is itself a finding about how human behavior on Reddit is not well approximated by linear assumptions.

1. Introduction and Problem Statement

1.1 Why This Question

Reddit is, by volume, one of the largest text-based social platforms in the world. Unlike feed-driven networks, Reddit is organized into thousands of community-defined subreddits, each with distinct norms about what counts as good content. A handful of posts in each community become viral — drawing thousands of upvotes and hundreds or thousands of comments — while the great majority receive almost no attention at all. Understanding the difference matters to multiple stakeholders. Content creators benefit from knowing which timing, framing, and tone choices increase their odds of being heard. Community moderators benefit from understanding how language and sentiment shape disagreement and engagement on their sub. Platform strategists benefit from any signal that helps surface high-quality content earlier.

Our project frames the problem with a single guiding question:

What makes a Reddit post go viral — positivity, negativity, or controversy?

To answer this in a way that is both rigorous and tractable at scale, we decomposed the question into ten sub-questions across exploratory data analysis, natural language processing, and predictive modeling. Each of the ten was implemented as a standalone PySpark job that reads Parquet from S3, performs distributed computation on the cluster, and writes its results back.

1.2 The Ten Questions

The ten questions were defined in our project proposal and held fixed throughout the project. Three groups roughly match the three required analysis components. Exploratory data analysis covered Questions 1 through 4. Natural language processing covered Questions 5, 6, and 7. Machine learning covered Questions 8, 9, and 10. The questions were chosen so that the answers from earlier questions feed into later ones — for example, the controversy label produced by Question 7 (sentiment variance across comments) is used as the target for Question 10's classifier.

Exploratory Data Analysis

- Q1. Which subreddits have the highest virality rate, both in absolute terms and relative to a subreddit's own baseline?
- Q2. What time of day, day of week, and posting cadence correlate with viral posts?
- Q3. How is engagement distributed across posts? Does it follow a power law?
- Q4. Does a user's posting history relate to virality? Do power users dominate or underperform?

Natural Language Processing

- Q5. How does post sentiment (positive, negative, neutral) correlate with engagement?
- Q6. What words distinguish viral posts from average posts (TF-IDF analysis)?
- Q7. Is there a controversy signal in the language itself? We measure sentiment variance across the comments on a post as a quantitative measure of disagreement.

Machine Learning

- Q8. Can we predict the number of comments a post will receive based on metadata available at post time?
- Q9. Can we classify a post as viral vs. not-viral using only early features (title, author, timing)?
- Q10. Can we classify a post as controversial vs. consensus, using the sentiment-variance label from Q7 as ground truth?

2. Data and Infrastructure

2.1 The Dataset

Our source dataset is the public Reddit dataset hosted on AWS S3 at `s3://dsan6000-datasets/reddit/parquet/`, covering Reddit submissions and comments from June 2023 through July 2024. The dataset is approximately 446 GB on disk in compressed Parquet, and contains roughly 500 million submission rows and approximately 3 billion comment rows in total. Each row carries fields including the post or comment identifier, link identifier, author, subreddit, title, body, creation timestamp, score (upvotes minus downvotes), upvote ratio, comment count, and content moderation flags.

The dataset is partitioned by year and month. Partition pruning — where Spark only reads files matching a predicate on a partition column — was used heavily; for instance, single-month development runs read only the `yyyy=2023/mm=06/` subdirectory rather than scanning the full table.

2.2 Cluster Architecture

All work was performed on an Apache Spark 3.4.4 standalone cluster running on AWS EC2. The cluster consisted of one master node plus three worker nodes, all `t3.large` instances. The master ran the Spark driver and the standalone Spark Master daemon; each worker ran a Spark Worker daemon and made two cores and four gigabytes of executor memory available, for a total of six executor cores across the cluster. Jobs were submitted with `--executor-memory 4G --driver-memory 2G --total-executor-cores 6` unless otherwise noted.

S3 access was configured through the Hadoop S3A filesystem with the EC2 instance profile providing credentials, so no AWS access keys were embedded in code. The shared `SparkSession` factory in `src/common.py` applies these settings consistently across every job, and centralizes the S3 bucket path so that switching between buckets requires changing exactly one line.

2.3 Code Organization

The repository is structured as `src/eda` for Question 1 through 4, `src/nlp` for Question 5 through 7, `src/ml` for Question 8 through 10, with shared infrastructure in `src/common.py` and cluster setup scripts in `infrastructure/`. A `wakeup_cluster` shell script handles starting EC2 instances after a lab session, refreshing public IPs, and starting Spark daemons on the master and workers. This was important because AWS Academy lab sessions have a four-hour limit, and the cluster's public IPs change each time it restarts.

Each question's Python script follows the same pattern: build a `SparkSession` through the common factory, read Parquet from S3 with partition pruning where appropriate, perform the computation, write results to a partitioned CSV in the results directory, and stop the `SparkSession`. This consistency made it straightforward to add new questions and to debug issues by examining the shared common module.

3. Findings

We present findings grouped by theme rather than strictly in question order. The themes are: who posts what, when posts succeed, what language drives engagement, and what we can predict in advance. Each theme combines results from several questions to tell a coherent story.

3.1 Theme A — The Shape of Engagement on Reddit

Before discussing what makes a post go viral, it is worth establishing what virality even means on Reddit. Question 3 attempts exactly this characterization.

3.1.1 Engagement is winner-take-all (Q3)

The score distribution on Reddit is extreme. Across the full 14-month dataset:

- Approximately 48 percent of submissions in our June 2023 sample receive a score of exactly 1 — that is, only the author's own automatic upvote and nothing else.
- A further 5 percent receive a score of 0 (downvoted to neutral or never voted on).
- Posts in the 2 to 5 score bucket account for 18 percent of posts but only 1.1 percent of total score awarded across the platform.
- At the other extreme, posts scoring above 10,000 account for 0.037 percent of submissions but 17 percent of total score in our single-month sample. Across the full dataset this same top-bucket pattern persists.

These findings are visible in Table 1 below, and they have a single straightforward interpretation: Reddit is winner-take-all. The Pareto principle does not just apply — it understates what is happening. A vanishingly small fraction of posts captures a disproportionate share of attention, and the great majority of posts are effectively invisible.

Table 1. Score distribution, June 2023 sample (~25.5 million submissions).

Score bucket	Posts	% of posts	% of total score
1	12,290,088	48.19%	1.01%
2-5	4,671,417	18.32%	1.13%
6-10	1,732,747	6.79%	1.10%
0	1,231,592	4.83%	0.00%
101-500	1,178,580	4.62%	21.16%
1001-5000	163,678	0.64%	26.47%
501-1000	208,317	0.82%	11.97%
5001-10000	15,246	0.06%	8.61%
10001+	9,423	0.04%	17.07%

Across the full 14-month dataset of approximately 524 million posts, the same shape repeats: posts scoring above 1,000 represent 0.34 percent of submissions but capture roughly 44 percent of total score, while posts scoring above 10,000 represent 0.014 percent of submissions but still capture 12 percent. The implication for everything that follows is direct: when we ask why some posts go viral, we are asking why a tiny minority succeed. Predicting that minority correctly is genuinely difficult, and a model that always predicts 'not viral' would be right approximately 99 percent of the time. Any classifier we build needs to be evaluated against that baseline rather than against raw accuracy, which is why we lean on ROC-AUC throughout.

3.1.2 Volume does not equal influence (Q4)

Question 4 asked whether power users dominate engagement. The answer is a sharp no, with a striking pattern in who actually drives engagement on the platform.

The top posters by raw volume are almost entirely automated accounts. The single most prolific author across the full dataset, EchoJobs, posted 2,132,305 submissions and produced an average score of 1.0 — equivalent to no human attention at all. The next four most prolific accounts (GrownBrilliance, AutoNewsAdmin, AutoNewspaperAdmin, and similar) follow the same pattern: hundreds of thousands or millions of posts, near-zero average score, zero viral posts.

When we instead group authors by activity bucket and examine viral rates, the most engaged authors are not the top volume posters but the moderate-activity human users:

Table 2. Author activity vs. virality, full dataset.

Activity bucket	Unique authors	Total posts	Viral rate
100+ posts	538,288	241,665,323	0.715%
21-100 posts	3,061,684	124,135,264	1.074%
6-20 posts	8,596,057	88,679,128	0.883%
2-5 posts	16,566,540	49,297,012	0.696%
1 post	17,802,010	17,802,010	0.571%

The 21–100 posts bucket (about three million authors) has the highest viral rate at 1.07 percent, beating the heavy 100+ posts bucket by roughly 50 percent in relative terms. The interpretation is consistent with what we see in the language and timing analyses: Reddit rewards intentional, human posting, not raw volume. Spam-grade automated accounts produce huge volumes of low-quality posts that the community ignores or filters.

3.2 Theme B — When Posts Succeed (Q1, Q2)

3.2.1 Subreddit context dominates absolute virality (Q1)

Question 1 examined which subreddits have the highest virality rate, with virality defined two ways. The first, absolute, is whether a post crosses a hard engagement threshold (score $\geq 1,000$ or comments ≥ 100). The second, relative, is whether a post crosses $10\times$ its own subreddit's median score. The two definitions tell different stories, and the difference is itself the finding.

Under the absolute definition, the top subreddits by virality rate in our June 2023 sample include SaintMeghanMarkle (45.0 percent), shid_and_camed (35.1 percent), whenhe (33.1 percent), PoliticalCompassMemes (31.8 percent), MemeVideos (29.5 percent), and PublicFreakout (28.3 percent). On the full dataset, the absolute top includes user-pages and adult-content subreddits, where post counts are smaller but engagement per post is high.

Under the relative definition the picture changes character. Top relative-virality subreddits include MadeMeSmile (45.4 percent), tiktokhots (42.1 percent), TalesFromTheFrontDesk (48.6 percent on the full dataset), toolgifs (49.5 percent on the full dataset), and rarepuppers (47.1 percent). These are smaller, more focused communities where the median post is quiet but the upper tail of posts strongly outperforms the community's own baseline.

This distinction has a direct practical consequence. A community manager looking at the absolute leaderboard might conclude that drama and politics drive engagement; a content creator using the relative leaderboard would instead conclude that small passionate communities reward consistent contributors. Both are correct in their own framing, and any single-number ranking of 'viral subreddits' is therefore misleading.

3.2.2 Timing matters but not as much as community (Q2)

Question 2 examined whether posting hour, day of week, and posting cadence correlate with viral rate. The pattern is clear and small. Across the full dataset:

- Viral rate by hour ranges from approximately 1.01 percent at 6 UTC (the dead-zone for North American audiences) up to approximately 1.79 percent in the late-afternoon UTC band (12–18 UTC, which corresponds to mid-morning through early afternoon on the U.S. East Coast).
- Viral rate by day of week ranges from 1.28 percent on Mondays to 1.48 percent on Thursdays, with Friday close behind at 1.44 percent.

The takeaway is that midweek afternoons (UTC) outperform weekends and overnight hours, but the multiplicative gap is well under two-to-one. Timing is real but small; community choice is much larger. A post on the wrong subreddit at the right time will still under-perform a post on the right subreddit at the wrong time. This is a common pattern in social-media data: post timing matters at the margin, but the structural choice (which community, what audience) dominates.

3.3 Theme C — What Language Drives Engagement (Q5, Q6, Q7)

3.3.1 Negative content gets more comments, on average (Q5)

Question 5 used a Spark NLP sentiment pipeline (DocumentAssembler → Tokenizer → Normalizer → Vivekn sentiment) to label each post by its title sentiment, then aggregated engagement metrics by sentiment label. The results, on the June 2023 sample, are clear:

Table 3. Sentiment vs. engagement (June 2023, sampled).

Sentiment	Posts	Avg score	Avg comments	Viral rate
Negative	91,377	50.74	7.67	1.495%
Positive	86,783	42.76	6.66	1.234%
Neutral / na	21,780	34.03	5.72	1.111%

Negative-sentiment posts have a higher average score (50.7 vs. 42.8), more comments per post (7.67 vs. 6.66), and a higher viral rate (1.50 percent vs. 1.23 percent) than positive-sentiment posts. Neutral content under-performs both.

The interpretation needs care. Reddit, like most engagement-driven platforms, has long been observed to reward emotional and especially negative content with comment volume. Comments are a stronger response than a quiet upvote, so a post that prompts argument or commiseration accumulates more engagement than one that elicits a passive 'I agree.' That dynamic matches what we see here. However, the second result from Q5 — sentiment by subreddit — shows that the negative-positive ranking is not universal. AskReddit posts skew negative in raw counts (697 negative, 521 positive in the top sample); karma-farming subreddits skew positive; NSFW subreddits split roughly evenly.

In other words, sentiment is community-specific. The aggregate finding that negative content out-engages positive content is real, but it is the average over a heterogeneous distribution. A creator on r/MadeMeSmile would not benefit from posting negative content, even though a creator on r/AskReddit might.

3.3.2 Viral and average posts use different vocabulary (Q6)

Question 6 used Spark MLlib's CountVectorizer and IDF transformers to compute TF-IDF scores separately on viral posts and on average posts, then surfaced the words with the highest TF-IDF scores in each group. The output is informative because it is contrastive — words like 'a' or 'the' have low TF-IDF in both groups and do not appear; words appear because they are distinctively common in one group relative to the other.

Top viral-group words in the full-dataset run included: people, one, like, think, best, favorite, game, pride, new, 2024, never. Top average-group words included: layerzero, token, airdrop, fun, looking, anyone, want, need, available, help.

The contrast is striking. Viral vocabulary is conversational and opinion-asking — 'people,' 'think,' 'favorite,' 'best,' 'one' — and event- or community-anchored — 'pride,' '2024,' 'game.' Average-post vocabulary is transactional and self-promotional — 'token,' 'airdrop,' 'available,'

'looking' — words that strongly suggest cryptocurrency promotion and karma-farming requests. This is consistent with what Q4 told us about volume posters: a large share of low-engagement content on Reddit is automated promotion that the community filters out.

The practical implication for a content creator is that titles which open conversation (asking opinions, naming a category, referencing a shared event) outperform titles which announce a transaction or request a favor. This is a vocabulary playbook visible directly from the data.

3.3.3 Sentiment variance as a controversy signal (Q7)

Question 7 introduced a custom controversy metric that does not depend on Reddit's own controversiality flag. For each post with at least five comments, we compute the variance of comment sentiment scores across the comments. A post on which most commenters are positive (or most are negative) has low variance; a post that splits the room — some commenters strongly positive, others strongly negative — has high variance. We label posts with sentiment variance ≥ 0.25 as controversial.

This question was implemented as a Spark NLP pipeline on a 15-day window of the June 2023 comments (June 1 through June 15). The pipeline runs DocumentAssembler, Tokenizer, Normalizer, and the pretrained Vivekn sentiment model on every qualifying comment, maps the resulting label to a numeric value (positive = 1, negative = 0), and aggregates per post. The 15-day window was chosen because the full-month run had previously exhausted the four-hour AWS Academy lab session before completing inference.

The Q7 inference completed within the lab budget, and the per-post variance distribution was successfully written to the cluster. We use the resulting label as ground truth for the Q10 classifier (described in section 3.4.3), where it functions as a target variable rather than as a stand-alone EDA result. From a methodological standpoint, sentiment variance is a useful operationalization of controversy because it is computed from the comments themselves rather than from any platform-level flag, and it captures the intuitive notion that a controversial post is one that divides its commenters.

3.4 Theme D — What We Can Predict (Q8, Q9, Q10)

The three machine-learning questions ask, in increasing order of difficulty: how many comments will a post get (Q8, regression), will it go viral (Q9, classification), and will it be controversial (Q10, classification)?

3.4.1 Predicting comment count is hard at scale (Q8)

Question 8 trained Spark MLlib regression models — Linear Regression, Random Forest Regressor, and Gradient Boosted Trees — to predict the number of comments a post will receive given posting hour, day of week, title length, subreddit (categorical), and other engineered features. We trained on the June 2023 sample first, then on the full dataset:

Table 4. Q8 regression results.

Model	RMSE (June 2023)	R ² (June 2023)	RMSE (full)	R ² (full)
Linear Regression	52.74	0.197	32.42	0.009
Random Forest	54.77	0.134	30.73	0.110
Gradient Boosted Trees	54.58	0.140	32.02	0.033

Two things in this table are worth flagging honestly. First, on the single-month sample Linear Regression appears to win ($R^2 = 0.197$) — that was the result we presented in our mid-project deck. Second, on the full dataset that result reverses: Linear Regression's R^2 collapses to 0.009, while Random Forest holds up best at 0.110, and the absolute RMSE values across all three models drop substantially because the full distribution has different scale.

The collapse of Linear Regression at scale is itself a finding. The relationship between simple metadata (hour, day, title length, subreddit ID) and comment count is not linear over the full dataset; the heterogeneity across subreddits, seasonality, and the heavy-tailed nature of comment counts (Q3) all break the linear assumption. The single-month R^2 of 0.20 was somewhat optimistic precisely because the sample was small enough to fit a line through. Random Forest, which can split on subreddit identity and then fit local models within each region, recovers about half of that R^2 on the full data and does so much more honestly.

The general lesson is that approximately 10 to 20 percent of variation in comment counts is recoverable from post-time metadata alone. This is consistent with what we know about social media engagement: most of the signal lives in content (the title and body) and in the early dynamics of the comment thread itself, neither of which are available as features at the moment a post is created. A useful next step, which we discuss in Section 5, is to add NLP features from the title text as inputs to the regression.

3.4.2 Predicting virality (Q9)

Question 9 reformulates the problem as binary classification: will this post be viral (defined by the same threshold as Q1, score $\geq 1,000$ or comments ≥ 100) given features available at post time? We trained Logistic Regression and Random Forest on a 509,060-post sample (about 80 percent train, 20 percent test):

Table 5. Q9 viral classifier results (June 2023 sample).

Model	ROC-AUC	F1	Accuracy
Logistic Regression	0.7136	0.7330	0.5947
Random Forest	0.7300	0.9795	0.9863

These numbers need a careful read. The Random Forest's accuracy of 98.6 percent and F1 of 0.98 sound spectacular but are misleading: the positive class (viral posts) is approximately

1 to 2 percent of all submissions, so a model that always predicts 'not viral' would already achieve 98+ percent accuracy. The metric that separates real signal from class-imbalance illusion is ROC-AUC, which measures discriminative power across all thresholds and is invariant to base rates.

On ROC-AUC, both models score in the 0.71 to 0.73 range. That is meaningfully above 0.5 (chance) and is a real predictive signal — but it is far from the 0.98 the F1 number might suggest. Random Forest edges Logistic Regression by about 2 ROC-AUC points, consistent with Q8: tree models recover non-linear interactions (subreddit × hour, for example) that the logistic model cannot.

In plain terms: from the metadata available at the moment a post is created, we can rank posts by their probability of going viral much better than chance, and the top-ranked decile of posts will contain a meaningful concentration of the actual viral posts. We cannot, however, predict virality precisely on a per-post basis — much of the variance lives in factors not visible in the metadata.

3.4.3 Predicting controversy (Q10)

Question 10 uses the controversy label produced by Q7 (sentiment variance ≥ 0.25 over post comments) as ground truth for a binary classifier. We used the Reddit built-in 'controversiality' flag on comments as the labeling signal, defining a post as controversial if at least 20 percent of its comments are flagged controversial by Reddit's own algorithm — an alternative operationalization that does not depend on running NLP inference. After applying a minimum-comments filter, the resulting labeled set contained 33,744 posts (26,916 train / 6,828 test), of which only 3.1 percent were controversial.

Table 6. Q10 controversy classifier results (June 2023 sample).

Model	ROC-AUC	F1
Logistic Regression	0.6456	0.9561
Random Forest	0.5940	0.9366

Again, F1 is inflated by class imbalance. The honest read is the ROC-AUC: 0.65 for Logistic Regression and 0.59 for Random Forest. Both models recover meaningful but modest signal — better than random, but well below virality prediction. There is also a methodological note in the Random Forest run: a warning was emitted reducing maxBins from 600,000 to 26,916 (the number of training instances), meaning that the high-cardinality subreddit identifier could not be split optimally and the tree model effectively had access to less information than the logistic model.

Comparing Q9 and Q10 directly is informative. Virality (Q9) is more predictable than controversy (Q10): ROC-AUC 0.73 vs. 0.65. Some of that difference is real and substantive — controversy is intrinsically harder to predict because it depends on the conversation that develops in the comments, which is not visible at post time. Some is also methodological: the Q9 viral label is defined by score and comments, both of which are correlated with metadata

like subreddit; the Q10 controversy label is defined by comment-level dynamics that may have weaker hooks into pre-comment metadata.

4. Cross-Question Synthesis

Reading the ten questions together yields a more coherent answer to our central question than any single question gives in isolation. Three integrated takeaways stand out.

4.1 Virality is multiplicative, not additive

None of our individual features — sentiment, timing, subreddit, author activity, vocabulary — produces a large effect on its own. Sentiment moves viral rate from about 1.1 percent (neutral) to about 1.5 percent (negative). Timing moves it from about 1.0 percent (worst hour) to about 1.8 percent (best hour). Author activity moves it from 0.6 percent (one-time posters) to 1.1 percent (moderate-activity users). These are real effects, but each is small in isolation.

What explains the strong tail of engagement that Q3 documents must therefore be interactions between features. The right title (Q6 vocabulary) on the right subreddit (Q1) at the right time (Q2) with the right sentiment for that community (Q5) is the configuration that produces a viral post. This matches the ML results: tree models, which can represent feature interactions, consistently out-perform logistic regression on the harder problem (Q10), even though logistic does well on the easier one (Q9). Predicting virality is fundamentally a high-dimensional interaction problem, not an additive one.

4.2 Community context is the single largest variable

Across every theme, the strongest moderator of every other effect is the subreddit. Sentiment-engagement relationships flip sign between subreddits (Q5). Absolute and relative virality measures rank entirely different sets of communities at the top (Q1). Top-author dominance differs by subreddit (Q4). The implication for predictive modeling is that subreddit-aware features are essential, and that aggregating across the platform without conditioning on community will produce average effects that wash out the most useful local signal. Our tree models recover this by splitting on the subreddit identifier; our linear model cannot, which contributes to its poor performance on the full dataset.

4.3 The platform actively rewards content that prompts reply

The combined evidence from Q3 (engagement distribution), Q5 (sentiment-engagement), Q6 (vocabulary), and Q9-Q10 (predictability gap between virality and controversy) suggests that Reddit's engagement structure rewards content that prompts a response. Posts that ask opinions get more comments. Negative posts (which often imply complaint or critique to be answered) get more comments. Posts on debate-oriented subreddits get more comments. Controversy — which by definition produces split responses — drives heavy comment threads even when score is muted. The platform's engagement mechanics reward conversational, replyable content over passive consumption, which is consistent with Reddit's threaded-discussion design.

5. Limitations and Honest Caveats

5.1 Class imbalance and metric choice

Our two binary classifiers (Q9 and Q10) report very high F1 and accuracy numbers (0.94 to 0.99) that do not reflect their actual discriminative power. The viral and controversial classes are each a few percent of the data, so a trivial 'always predict not-viral' baseline already hits 98+ percent accuracy. ROC-AUC is the metric that actually matters here, and we report 0.65 to 0.73 across our models — modest but real signal. A reader who only saw the F1 row in our results table would substantially overestimate model quality. We have flagged this throughout the report and would recommend any future iteration of these models use balanced precision-recall curves and PR-AUC alongside ROC-AUC.

5.2 Linear regression collapse at scale

The Q8 Linear Regression result on the full dataset ($R^2 = 0.009$) is much worse than on the June 2023 sample ($R^2 = 0.197$). This is not a bug; it is a real consequence of the heterogeneous, heavy-tailed distribution of comment counts across the full 14 months. Linear assumptions break down when the underlying relationship is not linear, and the full-dataset result is the correct one to take seriously. Random Forest is the model we would actually deploy.

5.3 Sentiment model selection

Our NLP pipeline used the Vivekn sentiment model from Spark NLP for all sentiment analysis, including in Q5 and Q7. Vivekn is a fast, lightweight model that operates on a binary positive-negative polarity. A more sophisticated model (a fine-tuned BERT-class model, for example) would produce richer features and potentially stronger downstream classifiers. Our project deck mentions BERT-based SentimentDL, but the actual production runs used Vivekn for performance reasons. We note this honestly: any production deployment of these findings should re-run with a stronger sentiment model.

5.4 Window selection for Q7

The Q7 controversy-signal pipeline was implemented on a 15-day window of June 2023 (the first half of the month) rather than a full month, because Spark NLP inference on a full month of comments exceeded the four-hour AWS Academy lab session limit. A full-month or full-dataset run would be more authoritative; the 15-day result is, however, sufficient for the purpose of generating a controversy label for use as Q10's ground truth, and the methodological structure scales straightforwardly to a longer window if a longer cluster session is available.

5.5 Engagement-velocity features were not added

The strongest predictor of whether a post will go viral is almost certainly the engagement it accumulates in the first 30–60 minutes of its life — comment velocity, early upvote rate, etc. We deliberately excluded these features from Q9 because the question asks what we can

predict at post time, not after the fact. A model that includes early-engagement features would substantially outperform our results, but it would also be answering a different question.

6. Recommendations

6.1 For content creators

Post on midweek afternoons (UTC), favor opinion-asking and community-celebratory titles, and choose subreddits where your content's sentiment matches the community norm. Negative-affect content out-engages positive in debate-oriented subreddits but underperforms in wholesome ones. The single largest decision is community choice; everything else is at-the-margin optimization.

6.2 For platform strategists

The combination of an extreme power-law engagement distribution (Q3) and a substantial population of automated low-engagement accounts (Q4) means that surfacing-mechanisms are the single most important lever for content discovery. Most posts die from obscurity rather than from poor quality. A real-time virality prediction service — even one with our modest 0.73 ROC-AUC — would be useful for early surfacing of likely high-engagement posts before they fall off the new queue.

6.3 For future work in this project line

- Add NLP-derived features (title sentiment, title TF-IDF embeddings) as inputs to the Q8/Q9/Q10 models. Q8 in particular is likely to gain the most from this.
- Re-run Q5 and Q7 with a transformer-based sentiment model (RoBERTa or BERT fine-tuned for social media) to capture nuances Vivekn misses.
- Build subreddit-specific models rather than a single platform-wide model. The cross-question synthesis suggests that subreddit context dominates almost every effect we measured.
- Augment the Q9 viral classifier with early-engagement features (first-30-minute comment velocity, upvote rate) for a 'will-be-viral-soon' prediction service.
- Run the full pipeline on the complete 14-month dataset once a longer cluster session is available, particularly for Q5, Q6, and Q7, where the single-month or 15-day windows are the binding constraint on confidence.

Technical Appendix

A.1 Cluster configuration

- Apache Spark 3.4.4 standalone, deployed on AWS EC2 t3.large instances.
- 1 master + 3 workers; 2 cores and 4 GB executor memory per worker; total cluster compute = 6 executor cores and 12 GB executor memory.
- Driver runs on the master in client mode; spark-submit invocations specify --executor-memory 4G --driver-memory 2G --total-executor-cores 6.
- S3 access via Hadoop S3A filesystem (org.apache.hadoop.fs.s3a.S3AFileSystem) with com.amazonaws.auth.InstanceProfileCredentialsProvider — no AWS keys in code.
- Vectorized Parquet reader enabled (spark.sql.parquet.enableVectorizedReader = true).

A.2 Repository structure

The project repository is organized as follows:

- infrastructure/ — cluster setup scripts (setup-spark-cluster.sh, install_worker.sh, wakeup_cluster), key files, cluster configuration.
- src/common.py — shared SparkSession factory and S3 path constants used by every job.
- src/eda/ — Q1 to Q4 jobs (virality by subreddit, temporal patterns, score distribution, author activity).
- src/nlp/ — Q5 to Q7 jobs (sentiment vs. engagement, viral vocabulary, controversy signal via sentiment variance).
- src/ml/ — Q8 to Q10 jobs (comment count regression, viral classifier, controversy classifier).
- results/ — per-question CSV outputs, organized by question and dataset slice.
- notebooks/, docs/ — supplementary materials.

A.3 Per-question script reference

Question	Script	Output CSVs
Q1	src/eda/q1_virality.py	q1_virality_absolute / q1_virality_relative
Q2	src/eda/q2_temporal.py	q2_by_hour / q2_by_dayofweek
Q3	src/eda/q3_score_distribution.py	q3_score_distribution / q3_power_law
Q4	src/eda/q4_author_activity.py	q4_top_authors / q4_by_activity_bucket
Q5	src/nlp/q5_sentiment_engagement.py	q5_sentiment_engagement / q5_sentiment_by_subreddit
Q6	src/nlp/q6_viral_vocabulary.py	q6_viral_vocabulary

Question	Script	Output CSVs
Q7	src/nlp/q7_controversy_signal.py	q7_controversy_comparison / q7_controversial_subreddits
Q8	src/ml/q8_comment_regression.py	q8_regression_results
Q9	src/ml/q9_viral_classifier.py	q9_viral_classifier
Q10	src/ml/q10_controversy_classifier.py	q10_controversy_classifier_results

A.4 Spark NLP pipeline (Q5, Q7)

The Spark NLP sentiment pipeline used in Q5 and Q7 is constructed identically in both jobs:

- DocumentAssembler — wraps a text column in the document annotation type expected by downstream stages.
- Tokenizer — splits the document into word tokens.
- Normalizer — lowercases and strips punctuation; configured with `setLowercase(true)`.
- ViveknSentimentModel.pretrained() — applies a pretrained Vivekn polarity model that emits 'positive' or 'negative' labels per document.
- Pipeline — bundles the four stages and is fit-and-transformed against the input DataFrame, producing a column of sentiment annotations from which the label is extracted via `.getItem(0).getField('result')`.

Inference is fully distributed across the cluster's executors. The pretrained model is downloaded once and cached on each worker (under `~/cache_pretrained/`), so subsequent runs incur only the inference cost. Submitted with `--packages com.johnsnowlabs.nlp:spark-nlp_2.12:5.5.3`.

A.5 Spark MLlib model configurations

Q8 (regression):

- LinearRegression with default solver.
- RandomForestRegressor with `maxBins=600000` (to accommodate high-cardinality subreddit identifier), `numTrees=50`, `maxDepth=6`, `seed=42`.
- GBTRRegressor with `maxIter=20`, `maxDepth=5`, `maxBins=600000`, `seed=42`.

Q9, Q10 (classification):

- LogisticRegression with `maxIter=20`, `regParam=0.01`, applied to features scaled via StandardScaler (with `Mean=False`, with `Std=True`).
- RandomForestClassifier with `maxBins=600000`, `numTrees=50`, `maxDepth=6`, `seed=42`; applied to unscaled features (trees are scale-invariant).
- Train/test split: 80/20 with `seed=42`.
- Evaluators: BinaryClassificationEvaluator (areaUnderROC) and MulticlassClassificationEvaluator (f1).

A.6 Cluster operations

Several practical notes accumulated during the project that future teams may find useful:

- AWS Academy / Learner Lab sessions are limited to four hours, after which all EC2 instances stop. The `wakeup_cluster` script handles restarting instances, refreshing public IPs, updating `cluster-config.txt`, and starting Spark daemons.
- Public IPs of EC2 instances change after each stop/start; private IPs persist. SSH from the `bigdata-host` to the cluster nodes uses private IPs (172.31.x.x), which is faster and not affected by IP rotation.
- `$SPARK_HOME` is set in `~/.bashrc` on cluster nodes but is not loaded by non-interactive SSH sessions; commands run via `ssh host 'cmd'` must specify the full Spark path (`/home/ubuntu/spark/sbin/start-master.sh`, etc.) or source `~/.bashrc` explicitly.
- S3 read access via the EC2 instance profile is independent of the AWS Academy CLI credentials; the latter expire frequently but Spark continues to read S3 for the duration of the cluster session.
- Q7's Spark NLP inference completed within budget on a 15-day window; a full-month run timed out the lab session in earlier attempts. Future runs that need full-month NLP should plan around a longer cluster session or use checkpointing.

A.7 Repository link and code

All Spark code, infrastructure scripts, intermediate result CSVs, and this report are committed to the project's GitHub repository. The repository's commit history reflects the contributions of all three team members across infrastructure, EDA, NLP, and ML components.